

HTML ~> hyper text Markup language

↳ describes the structure of a web page.

↳ Tell HTML tell the browser how to display the content.

<!DOCTYPE html> ~> declaration defines that this document is an HTML5 doc

<html> ~> element is the root element of an HTML page.

<head> ~> element contain meta information about the HTML page

<title> Page Title </title> ~> element specifies a title for the HTML page

</head>

<body> ~> element defines the document's body, and is a container for all the visible contents, such as heading, paragraph, images etc

<h1> My first heading </h1>

<p> My first paragraph. </p>

</body>

</html>

Some html elements don't have an ending tag hence called empty elements. Empty elements don't have an end tag.

HTML links

 This is a link

HTML Images.

↳ alternate if img isn't available.

↳ src can take two types of URL

↳ size

↳ Absolute URL (usually externally hosted)

↳ Relative URL (usually self hosted)

`<b style="color:red;"> This is a red paragraph </b>`
↳ used for inline CSS

The lang Attribute

`<!DOCTYPE html>`

`<html lang="en">`

`<body>`

....

`</body>`

`</html>`

HTML Headings.

↳ html heading are defined with the `<h1>` to `<h6>` tags.

↳ `<h1>` defines the most important heading. `<h6>` defined the least important heading.

↳ These are important as search engines use the heading to index the structure and content of your web pages.

HTML Paragraphs

Week → 1 (JavaScript)

compiled language

Interpreted language

usually go line by line.

1) first need to compile, then need to run.

2) Usually don't compile if there an error in the code.

ex → java, c++

Can run partially if the error occurs later.

ex - js, python

Arrays

```
const personArray = ["harkirat", "raman", "priya"]
```

```
const genderArray = ["male", "male", "female"];
```

```
const numberOfUsers = personArray.length;
```

```
for (let i = 0; i < numberOfUsers; i++) {
```

```
  if (genderArray[i] === "male") {
```

```
    console.log(personArray[i]);
```

```
  }
```

```
}
```

JavaScript Objects →

```
const user1 = {
```

```
  firstName: "harkirat",
```

```
  gender: "male"
```

```
}
```


`user["firstName"]` ~ accessing Object Elements.

functions ↴ remember to use these quotation while accessing something.

```
function sum(a, b) {  
    const sumValue = a + b;  
    return sumValue;  
}
```

```
const value = sum(1, 2);  
console.log(value);
```

JavaScript Callbacks → Function as an Argument

```
function sum(num1, num2, fnToCall) {  
    let result = num1 + num2;  
    fnToCall(result);  
}
```

```
function displayResult(data) {  
    console.log("Result of the sum is: " + data);  
}
```

```
function displayResultPassive(data) {  
    console.log("Sum's result is: " + data);  
}
```


// you are only allowed to call one function after this
// how will you displayResult of a sum.

```
const ans = sum(1, 2, displayResult);
```

Another Type.

```
function calculateArithmetic(a, b, arithmeticFinalFunction) {
```

```
    const ans = arithmeticFinalFunction(a, b);
```

```
    console.log(ans);
```

```
}
```

```
function sum(a, b) {
```

```
    return a + b;
```

```
}
```

```
const value = calculateArithmetic(1, 2, sum);
```

```
console.log(value);
```

↳ will be discussed later in detail.

different / Various ways of creating function in javascript.

1) function greet(name) {

```
    return "Hello, " + name;
```

```
}
```

↳ hoisted

↳ is a JS default behavior of moving

declaration to the top of their scope (before code execution). Remember
declaration are hoisted not initialization.

2) Function Expression (Anonymous or Named)

```
const greet = function(name) {  
    return "Hi , " + name;  
};
```

↳ not hoisted & since const is there so ; is preferred.

3) Arrow Function (ES6+)

```
const greet = (name) => {  
    return "Hey , " + name;  
};
```

// or if single return

```
const greet = name => "Yo , " + name;
```

4. Immediately Involved Function Expression (IIFE)

```
(function(name) {  
    console.log("Welcome , " + name);  
})("Eve"); → // Welcome , Eve
```

Common JavaScript Web APIs

```
setTimeout(() => {  
    console.log("Runs after 2 seconds");  
}, 2000);
```

```
const intervalId = setInterval(() => {  
    console.log("Runs every 3 seconds");  
}, 3000);
```


// To stop the interval after 10 seconds.

```
setTimeout(() => clearInterval(intervalId), 10000);
```

// Networking API

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

- then(response => response.json())

- then(data => console.log("fetched data:", data));

JavaScript Callback ~

(function as an argument) ~ usually in javascript
an argument can be a string, boolean, integer or a
function

```
function square(a) {  
  return a * a;  
}
```

```
function sumOfSomething(a, b, fn) {  
  const val1 = fn(a);  
  const val2 = fn(b);  
  return val1 + val2;  
}
```

```
sumOfSomething(a, b, square)
```


Async Nature of Javascript

↳ Task Delegation to other threads

setTimeout, fs.readFile, etc

Promises

↳ with we can create an async function, basically a wrapper on top of JS provided async function.

Example → Un Promised Code

```
const fs = require('fs');  
  
function myAsyncFunc(callback) {  
  fs.readFile("a.txt", "utf-8", function(err, data) {  
    callback(data);  
  });  
}  
  
// callback function to call  
function onData(data) {  
  console.log(data);  
}  
  
myAsyncFunc(onData);
```

Self made Async func

Same Promised Code

```
const fs = require('fs');  
  
// my own synchronous function  
function myAsyncFunc() {  
  return new Promise(function(resolve) {  
    fs.readFile("a.txt", "utf-8", function(err, data) {  
      resolve(data);  
    });  
  });  
}  
  
// callback function to call  
function onDone(data) {  
  console.log(data);  
}  
  
myAsyncFunc().then(onDone);
```

Async / Await Syntax

↳ just better syntax still uses promises
and callbacks under the hood and usually used at the
callers side


```
function myAsyncFunction( ) {  
  return new Promise( Function( callback) {  
    // do some async logic here  
    callback( "hi there" );  
  } );  
}
```

```
async function main( ) {  
  const value = await myAsyncFunction( );  
  console.log( value );  
}  
main( );
```

await only works with async function wrapper.